

Try Boukheang

ID: 2024476

01. Role-Based User Management System Using Microservices API Gateway

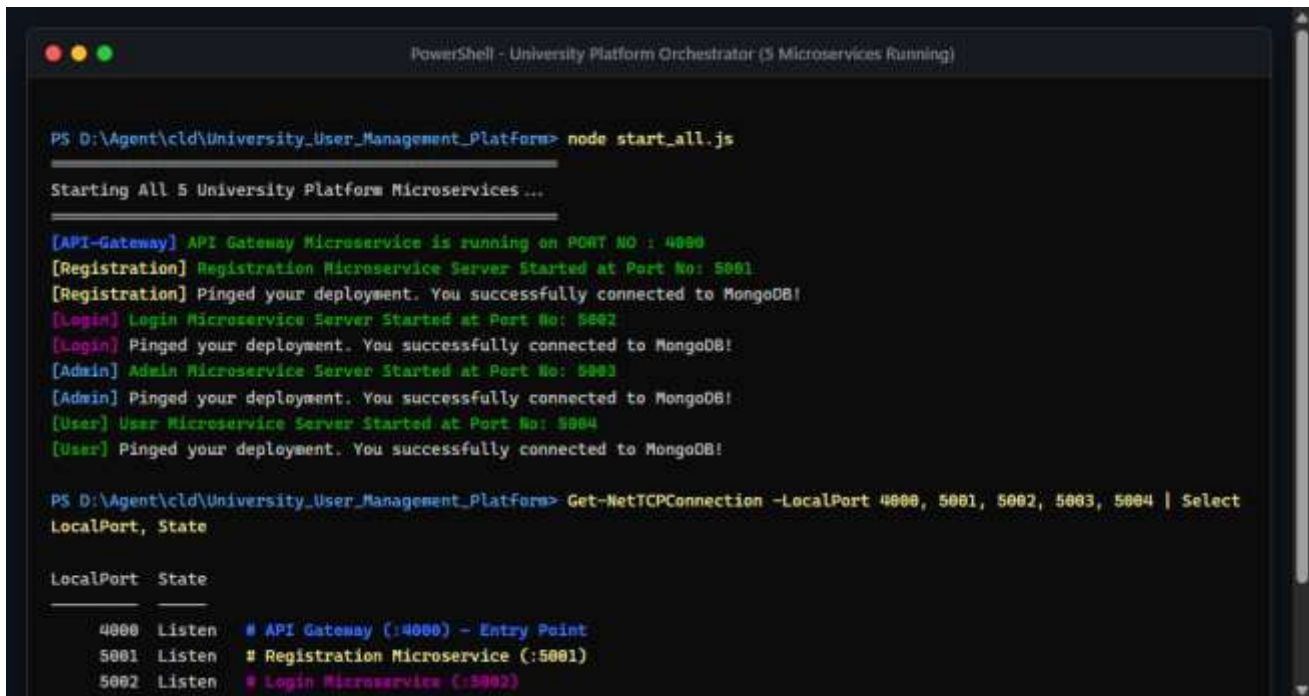
NodeJS MongoDB

Real-Life Scenario & System Architecture: A university Central Identity and User Management Platform developed as independent microservices communicating through an API Gateway with MongoDB Atlas persistence. Features two user classes: User (students/staff managing own profile) and Admin (administrators managing accounts). Strictly enforces mutual exclusion: Users can NEVER access Admin APIs, and Admins can NEVER access User APIs.

Microservice	Port	Endpoints	Role / Responsibilities
API Gateway	4000	All routes (/) via reverse proxy	Single entry point, JWT verification, role-based route guard
Registration	5001	POST /register/userregister	Validates input, checks duplicate email, bcrypt password hash, MongoDB store
Login (Auth)	5002	POST /auth/login	Validates email/password/role, issues signed 24h JWT token
Admin Service	5003	GET /admin/searchuser, viewalluser DELETE /admin/deluser	Admin operations: search by name/email, view all users, delete user
User Service	5004	GET /user/viewprofile PUT /user/updateprofile	User operations: view authenticated profile, update profile details

Task 1 & Task 2: 5 Microservices Setup & Database Connection Verification

Implementation Overview: Created 5 independent microservices (API Gateway, Registration, Login, Admin, and User services). Configured reusable dbconnect.js in all data-handling microservices establishing stable connection to MongoDB Atlas with connection pooling, ping health check verification, and environment variable configuration.



```
PowerShell - University Platform Orchestrator (5 Microservices Running)

PS D:\Agent\cld\University_User_Management_Platform> node start_all.js

Starting All 5 University Platform Microservices ...

[API-Gateway] API Gateway Microservice is running on PORT NO : 4000
[Registration] Registration Microservice Server Started at Port No: 5001
[Registration] Pinged your deployment. You successfully connected to MongoDB!
[Login] Login Microservice Server Started at Port No: 5002
[Login] Pinged your deployment. You successfully connected to MongoDB!
[Admin] Admin Microservice Server Started at Port No: 5003
[Admin] Pinged your deployment. You successfully connected to MongoDB!
[User] User Microservice Server Started at Port No: 5004
[User] Pinged your deployment. You successfully connected to MongoDB!

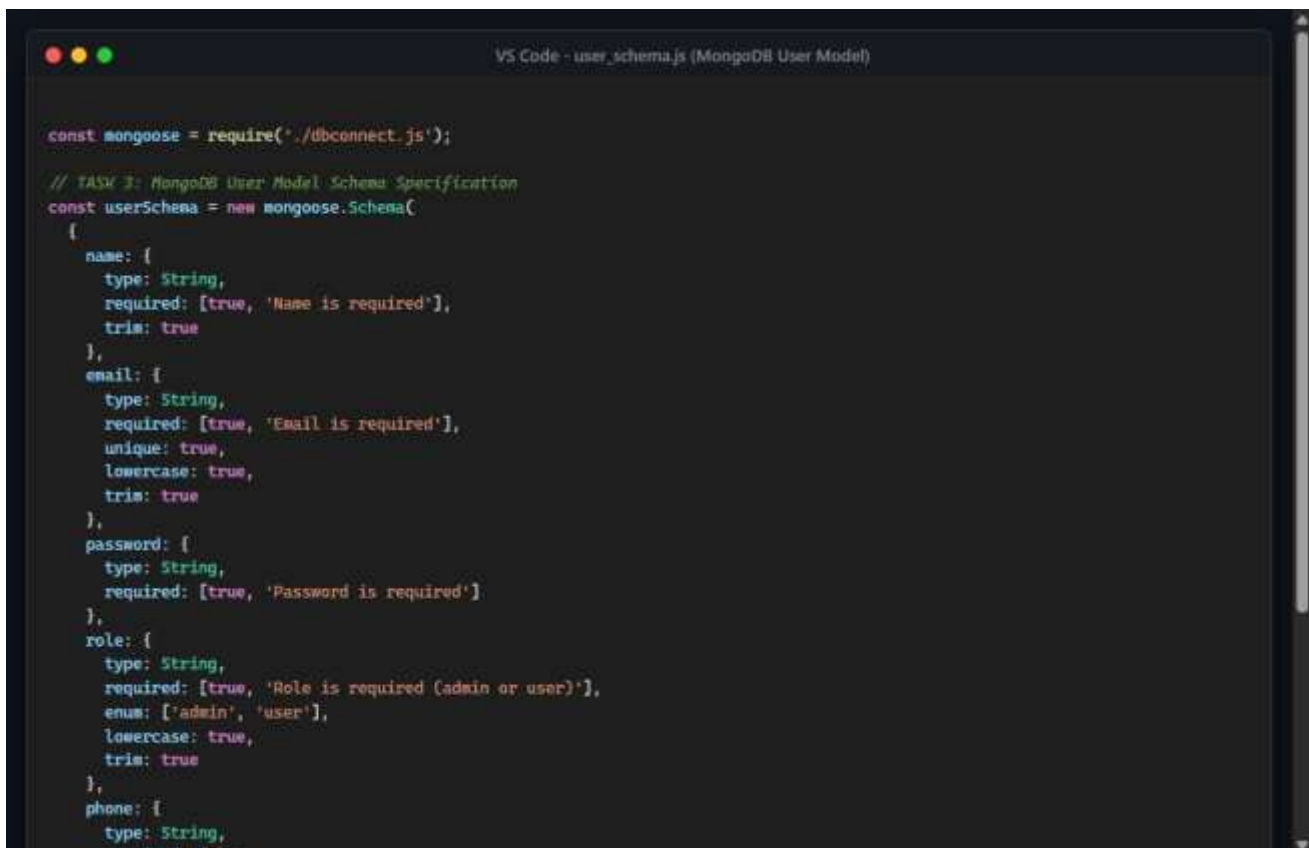
PS D:\Agent\cld\University_User_Management_Platform> Get-NetTCPConnection -LocalPort 4000, 5001, 5002, 5003, 5004 | Select
LocalPort, State

LocalPort State
-----
4000 Listen # API Gateway (:4000) - Entry Point
5001 Listen # Registration Microservice (:5001)
5002 Listen # Login Microservice (:5002)
```

Figure 1: PowerShell Orchestrator - All 5 Microservices running simultaneously on ports 4000, 5001, 5002, 5003, and 5004 with MongoDB Atlas connected.

Task 3: MongoDB User Model Implementation (user_schema.js)

Schema Specification: Designed and implemented the Mongoose User Model matching the exact schema requirements. Fields include `_id` (ObjectId), name, email (unique index, lowercase), password (bcrypt hashed), role (enum: 'admin', 'user'), phone, and automatic timestamps (createdAt, updatedAt).



```
VS Code - user_schema.js (MongoDB User Model)

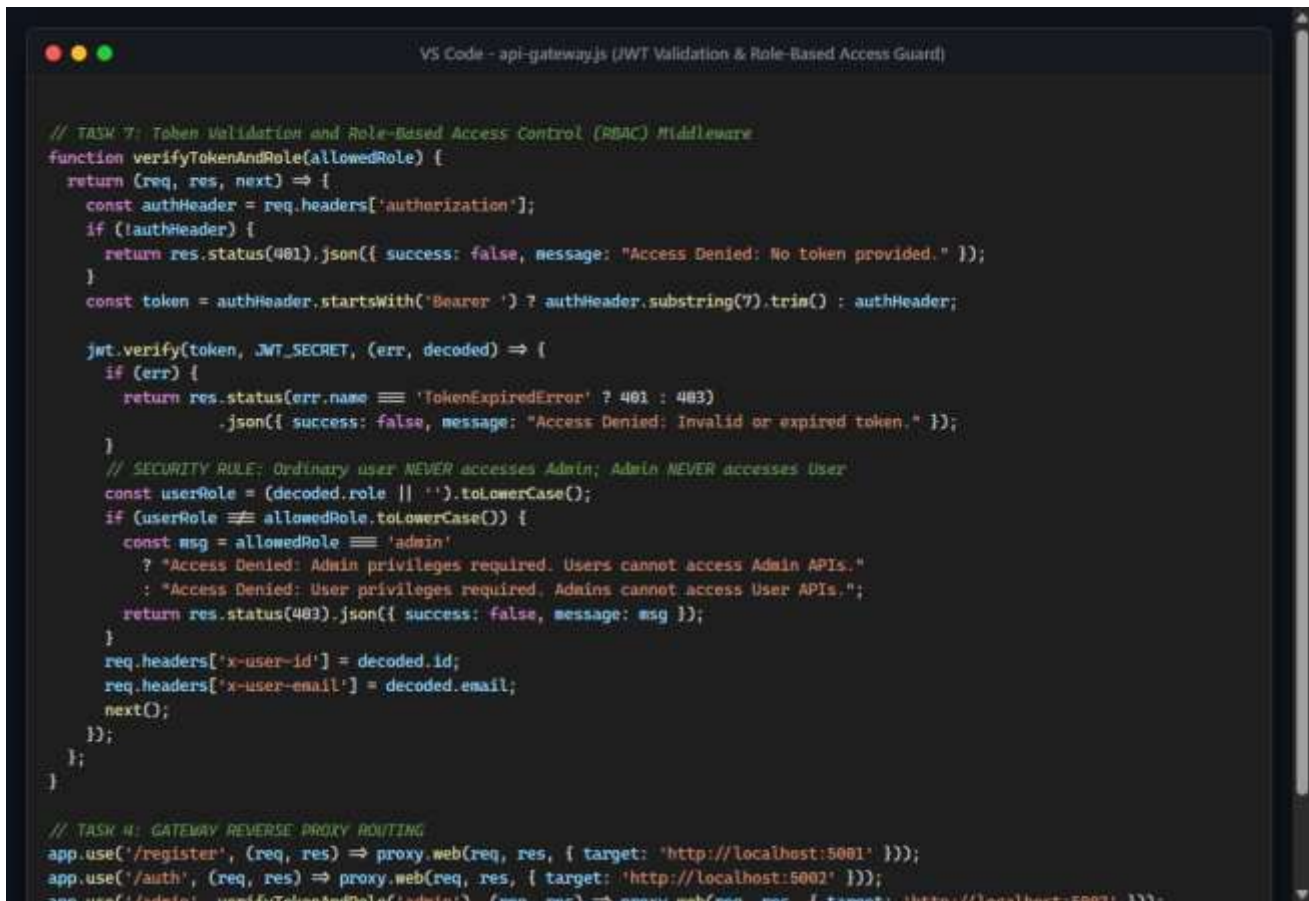
const mongoose = require('./dbconnect.js');

// TASK 3: MongoDB User Model Schema Specification
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true
    },
    password: {
      type: String,
      required: [true, 'Password is required']
    },
    role: {
      type: String,
      required: [true, 'Role is required (admin or user)'],
      enum: ['admin', 'user'],
      lowercase: true,
      trim: true
    },
    phone: {
      type: String,
```

Figure 2: VS Code Editor - Mongoose User Model Schema (user_schema.js) with required fields, validation, and timestamps.

Task 4 & Task 7: API Gateway Microservice & JWT Role-Based Routing Guards

Gateway & Security Architecture: The API Gateway acts as the single entry point for all client requests. Implements http-proxy reverse routing and strict JWT token verification middleware. Requests lacking a token or containing invalid/expired tokens are rejected. Crucially, role-based authorization enforces that ordinary users are blocked from Admin APIs (403 Forbidden) and administrators are blocked from User APIs (403 Forbidden).



```
// TASK 7: Token Validation and Role-Based Access Control (RBAC) Middleware
function verifyTokenAndRole(allowedRole) {
  return (req, res, next) => {
    const authHeader = req.headers['authorization'];
    if (!authHeader) {
      return res.status(401).json({ success: false, message: "Access Denied: No token provided." });
    }
    const token = authHeader.startsWith('Bearer ') ? authHeader.substring(7).trim() : authHeader;

    jwt.verify(token, JWT_SECRET, (err, decoded) => {
      if (err) {
        return res.status(err.name === 'TokenExpiredError' ? 401 : 403)
          .json({ success: false, message: "Access Denied: Invalid or expired token." });
      }
      // SECURITY RULE: Ordinary user NEVER accesses Admin; Admin NEVER accesses User
      const userRole = (decoded.role || '').toLowerCase();
      if (userRole !== allowedRole.toLowerCase()) {
        const msg = allowedRole === 'admin'
          ? "Access Denied: Admin privileges required. Users cannot access Admin APIs."
          : "Access Denied: User privileges required. Admins cannot access User APIs.";
        return res.status(403).json({ success: false, message: msg });
      }
      req.headers['x-user-id'] = decoded.id;
      req.headers['x-user-email'] = decoded.email;
      next();
    });
  };
}

// TASK 4: GATEWAY REVERSE PROXY ROUTING
app.use('/register', (req, res) => proxy.web(req, res, { target: 'http://localhost:5001' }));
app.use('/auth', (req, res) => proxy.web(req, res, { target: 'http://localhost:5002' }));
app.use('/admin', verifyTokenAndRole('admin'), (req, res) => proxy.web(req, res, { target: 'http://localhost:5003' }));
```

Figure 3: VS Code Editor - API Gateway source code (*api-gateway.js*) displaying JWT verification, role guard middleware, and microservice reverse proxy routes.

Task 5: Registration Microservice API & Database Verification

Registration Functionality: Registration API endpoint POST /register/userregister routed through API Gateway to Port 5001. Validates input, enforces unique email constraint, securely hashes passwords using bcrypt (10 rounds), and saves documents into MongoDB Atlas.

5.1: Student User Registration (Role: user)

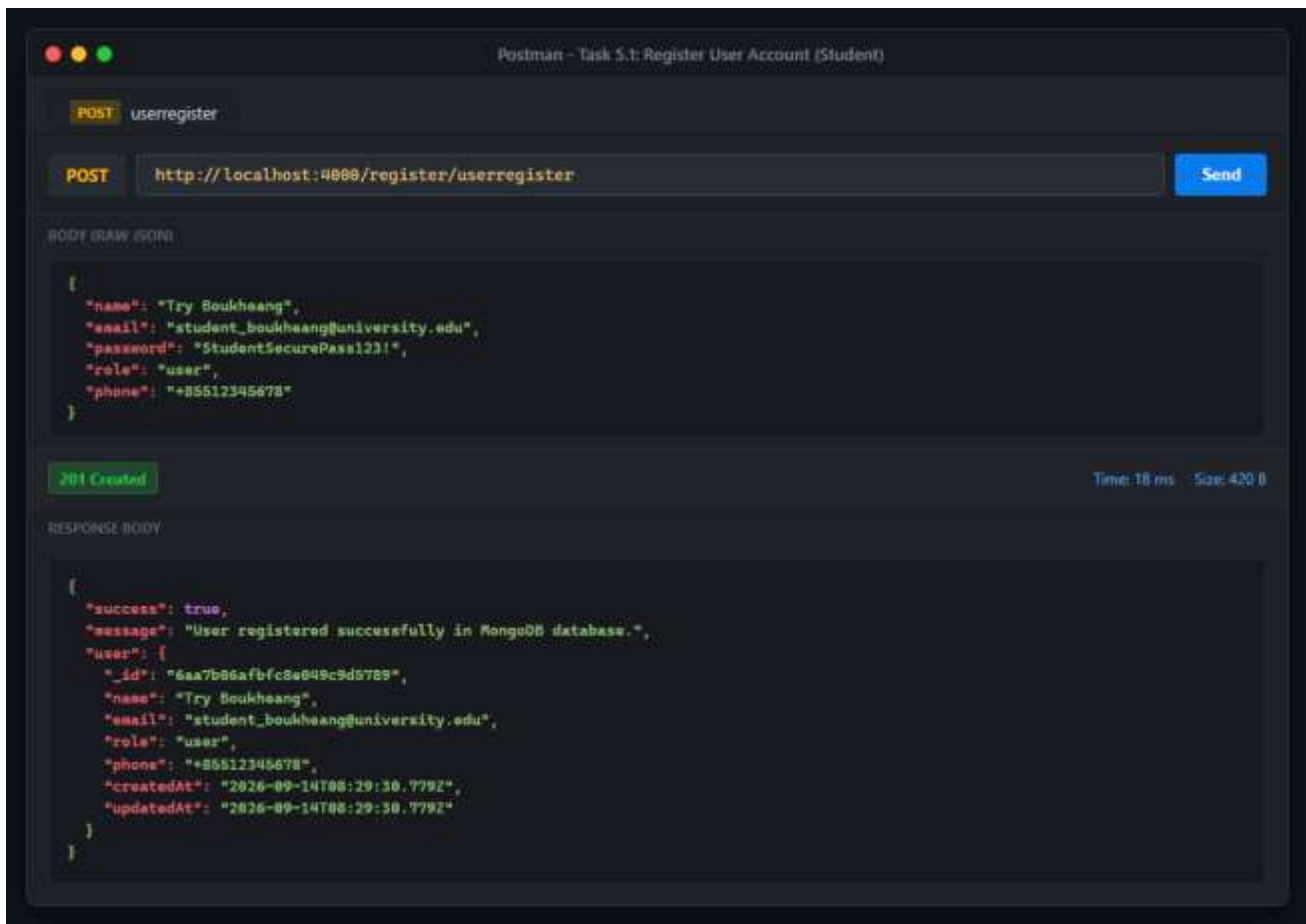


Figure 4: Postman - POST `http://localhost:4000/register/userregister` registering student account (Status 201 Created).

5.2: Administrator Account Registration (Role: admin)

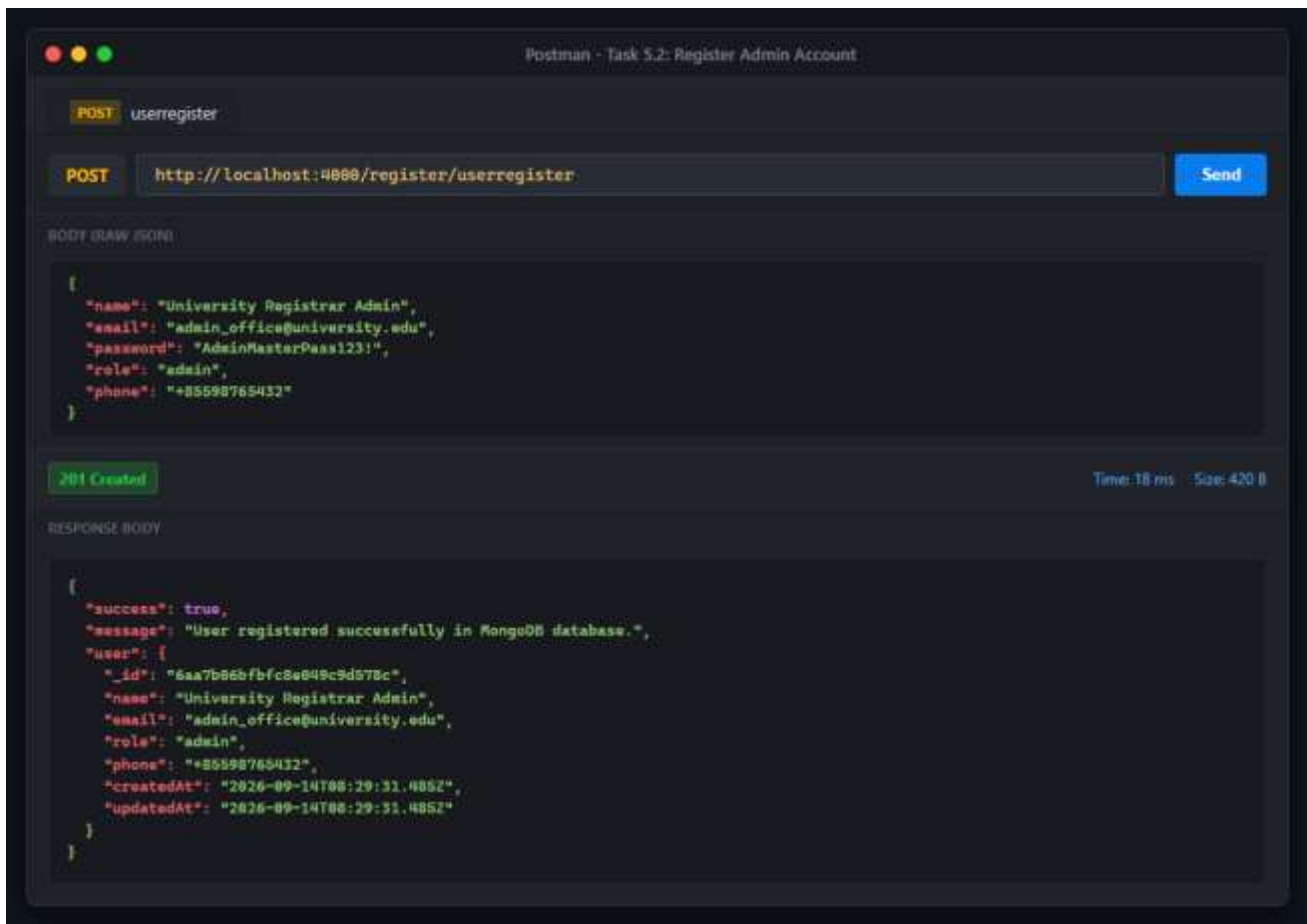


Figure 5: Postman - POST `http://localhost:4000/register/userregister` registering administrative account (Status 201 Created).

5.3: Duplicate Email Prevention Test

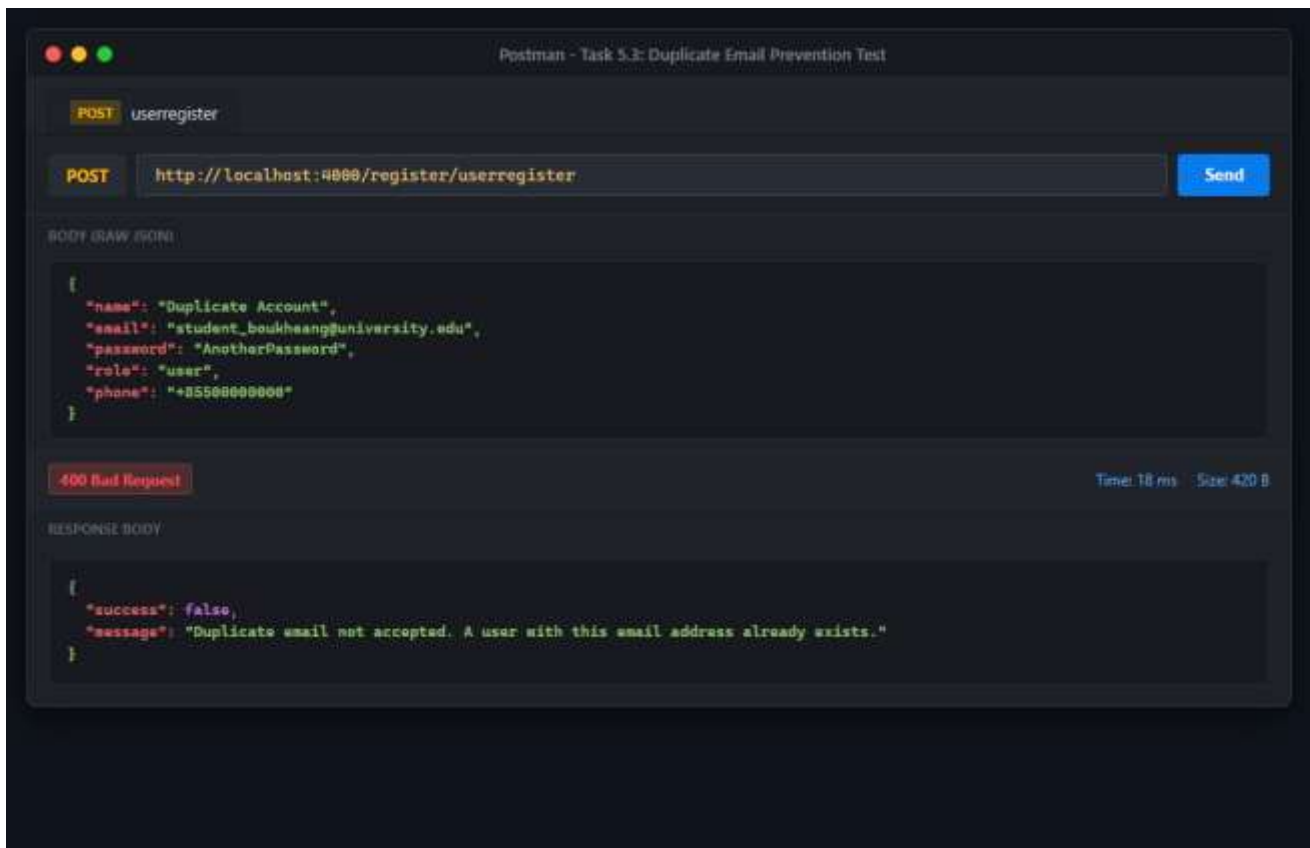


Figure 6: Postman - Re-registering with existing email rejected with Status 400 Bad Request ('Duplicate email not accepted').

5.4: MongoDB Atlas Database Verification

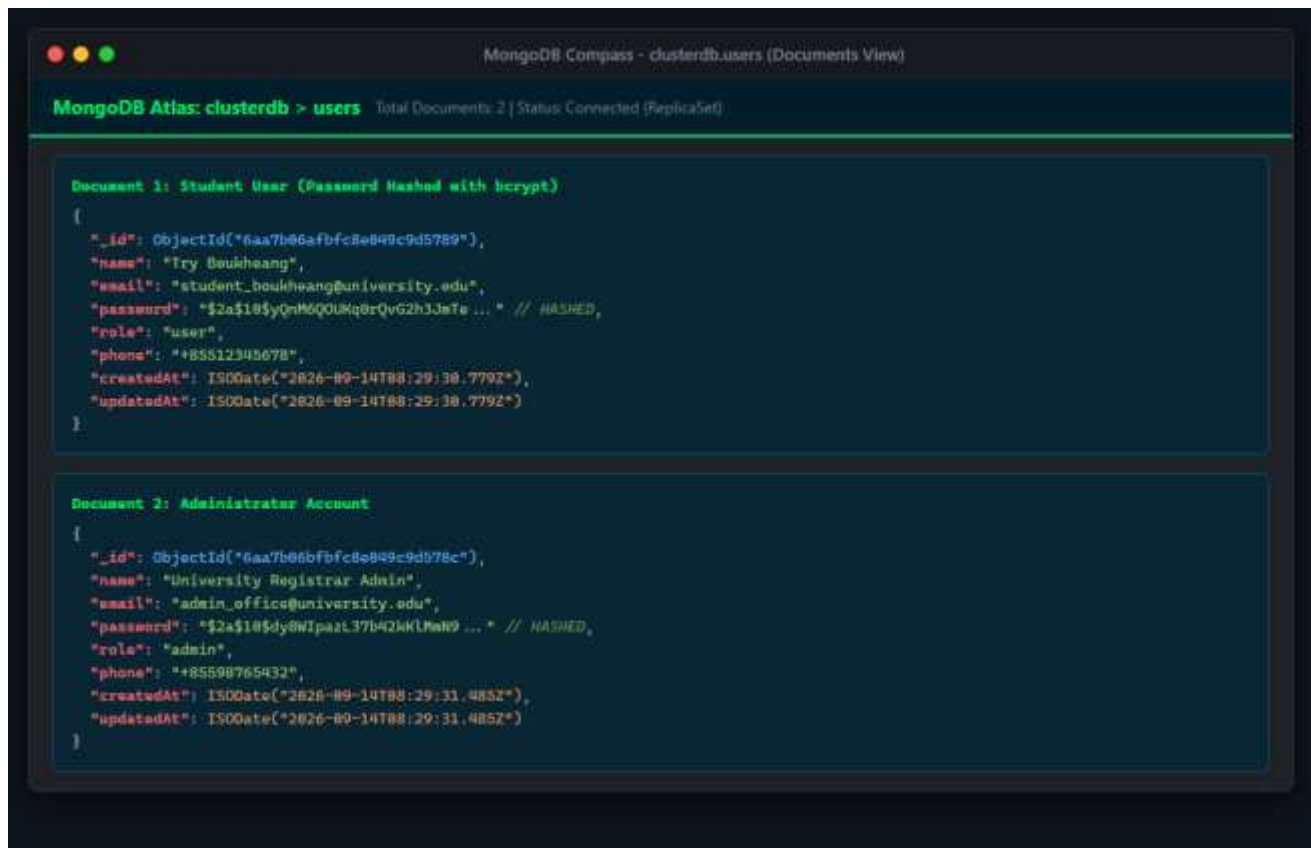


Figure 7: MongoDB Compass / Atlas Data View - Registered users stored in clusterdb.users showing bcrypt password hashes.

Task 6: Login Microservice API & JWT Token Issuance

Authentication Workflow: Login API endpoint POST /auth/login routed through API Gateway to Port 5002. Validates email, password (via bcrypt.compare), and requested role against MongoDB. Upon successful validation, issues a signed JWT containing user ID, email, and role valid for 24 hours.

6.1: Valid Student Login (Returns JWT)

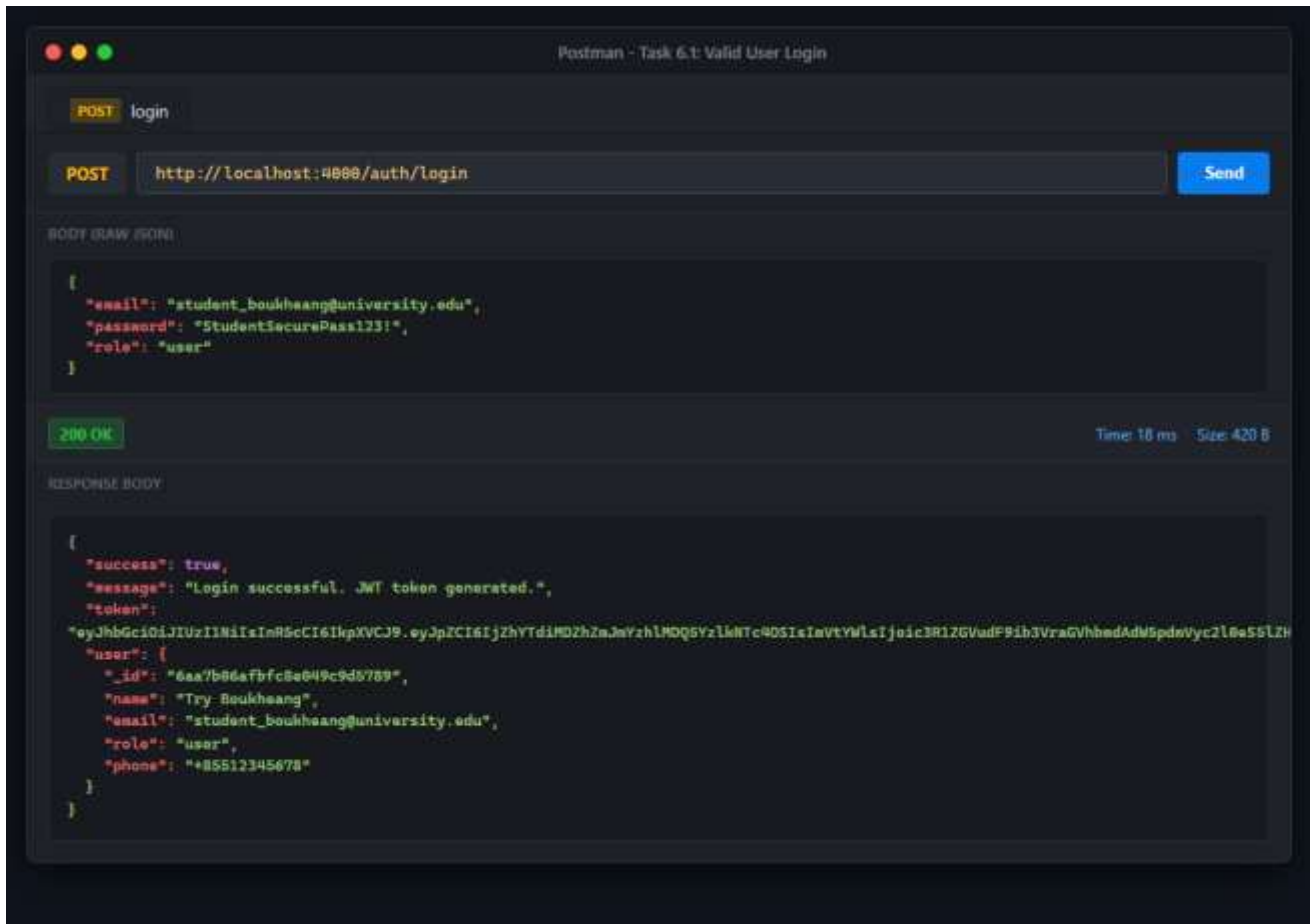


Figure 8: Postman - POST `http://localhost:4000/auth/login` for user returning 200 OK and signed JWT token.

6.2: Valid Administrator Login (Returns JWT)

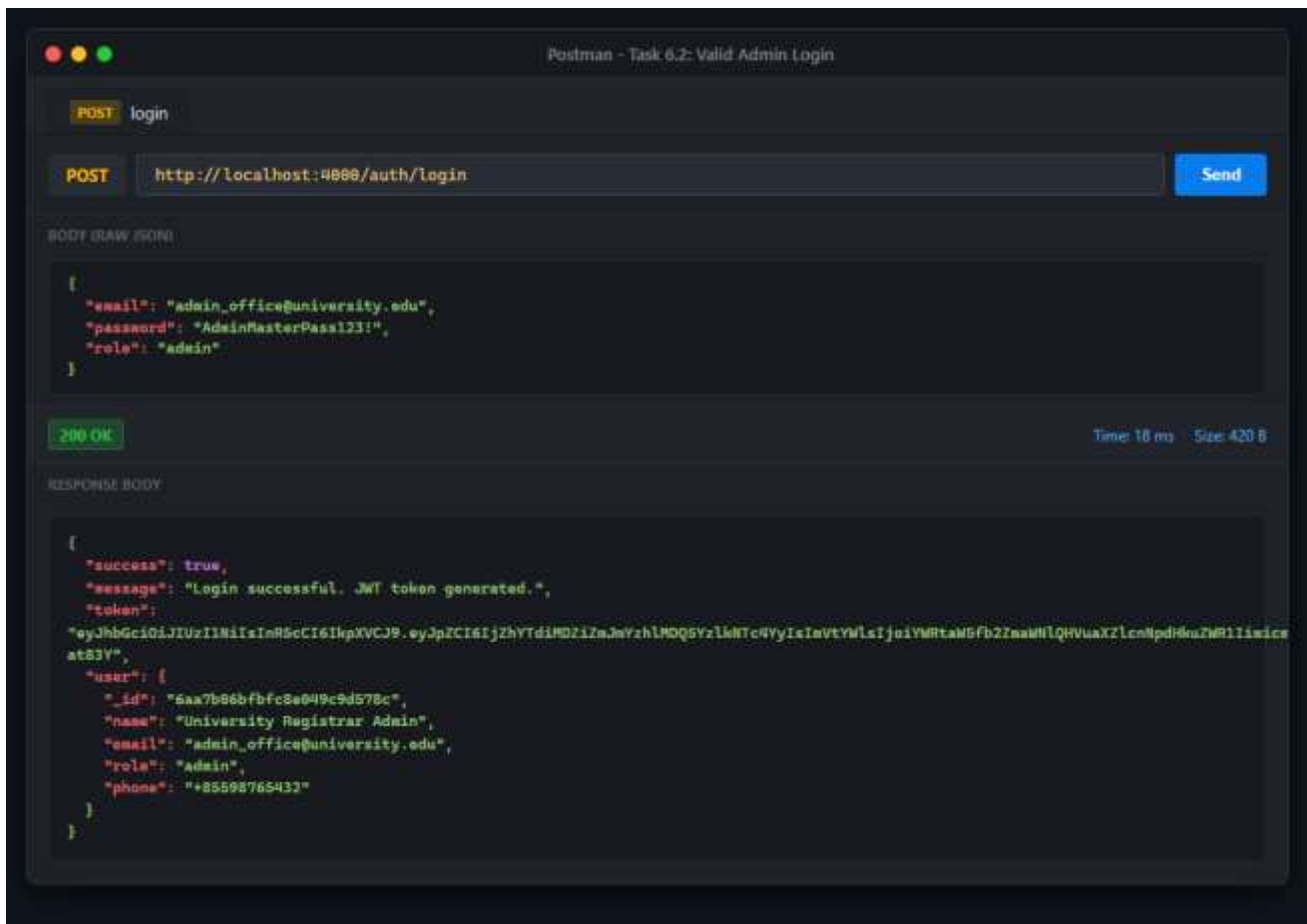


Figure 9: Postman - POST `http://localhost:4000/auth/login` for admin returning 200 OK and signed JWT token.

6.3: Login Attempt with Incorrect Password

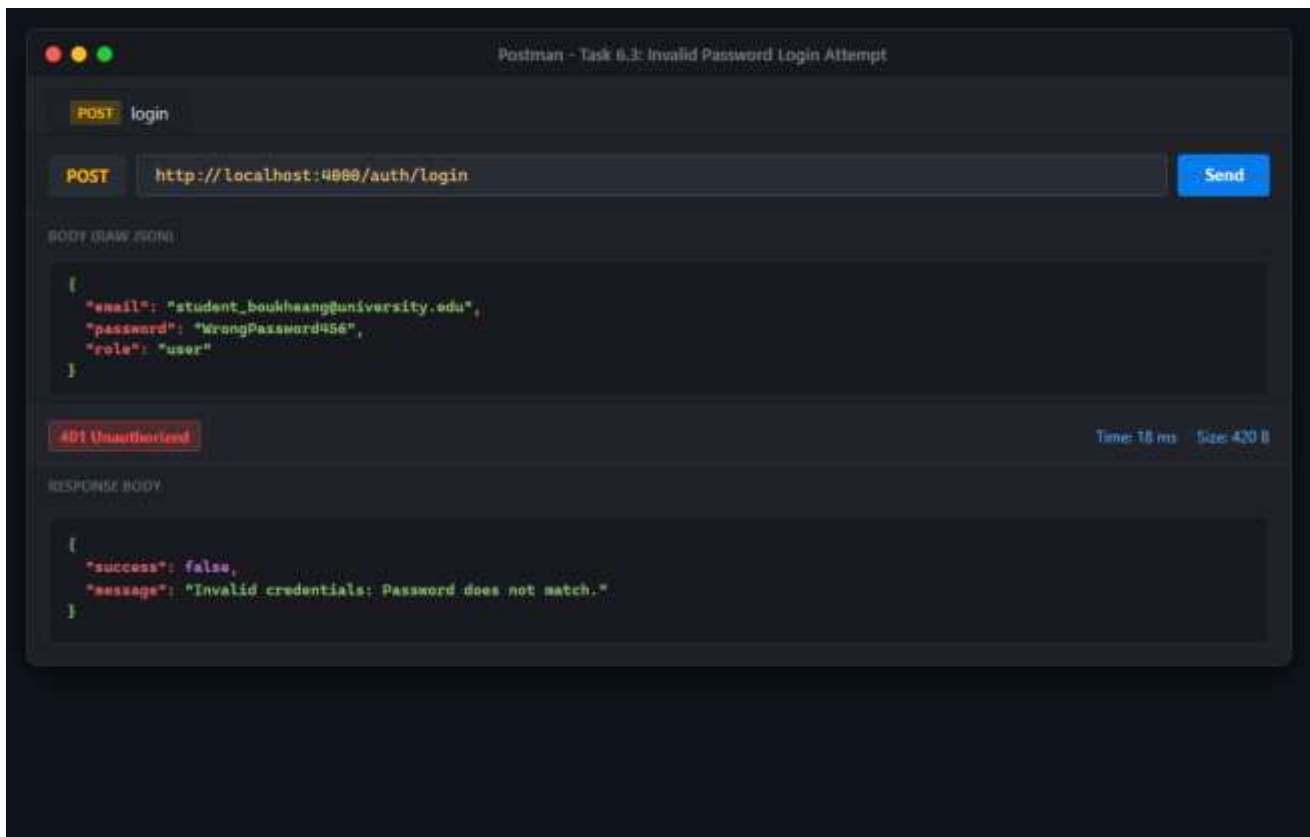


Figure 10: Postman - Incorrect password rejected with Status 401 Unauthorized ('Password does not match').

6.4: Login Attempt with Role Mismatch

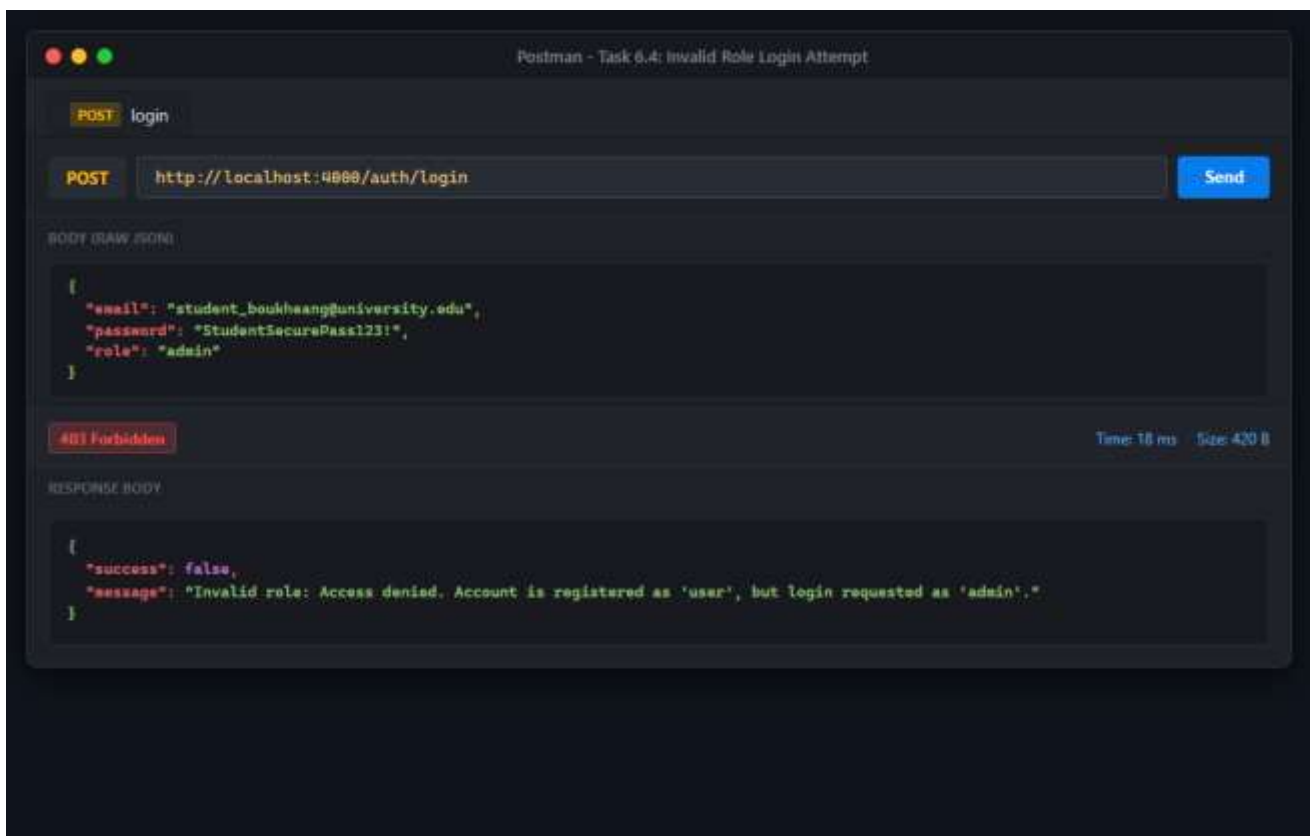


Figure 11: Postman - Student account attempting admin login rejected with Status 403 Forbidden ('Role mismatch').

Task 8: Admin Microservice APIs

Admin Capabilities: Admin Microservice (Port 5003) provides administrative user management capabilities. Access is strictly protected by the API Gateway requiring valid Administrator JWT tokens.

8.1: View All Users Information

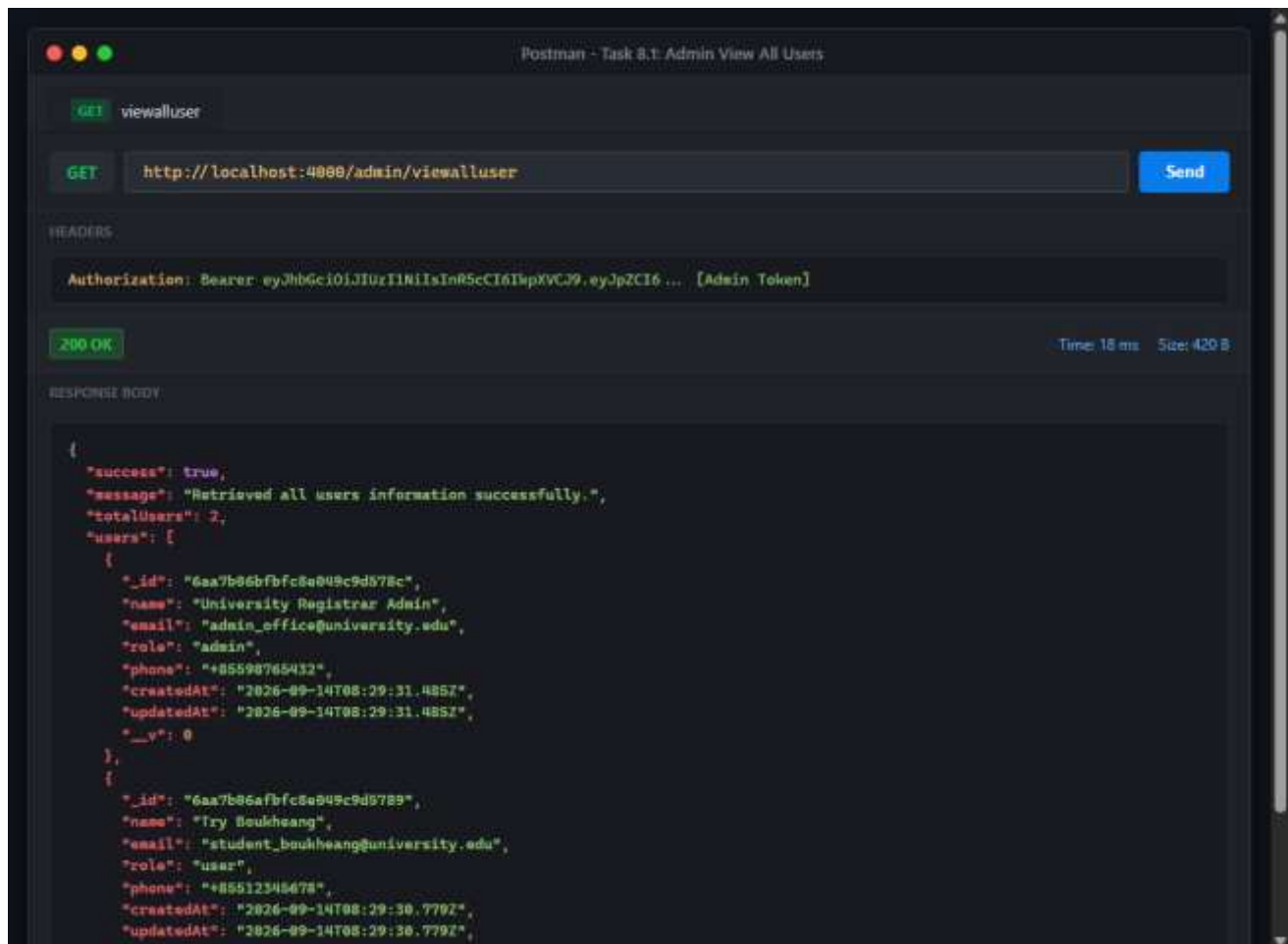


Figure 12: Postman - GET `http://localhost:4000/admin/viewalluser` with Admin Token returning all registered users (Status 200 OK).

8.2: Search User by Query Parameter (Found vs Not Found)

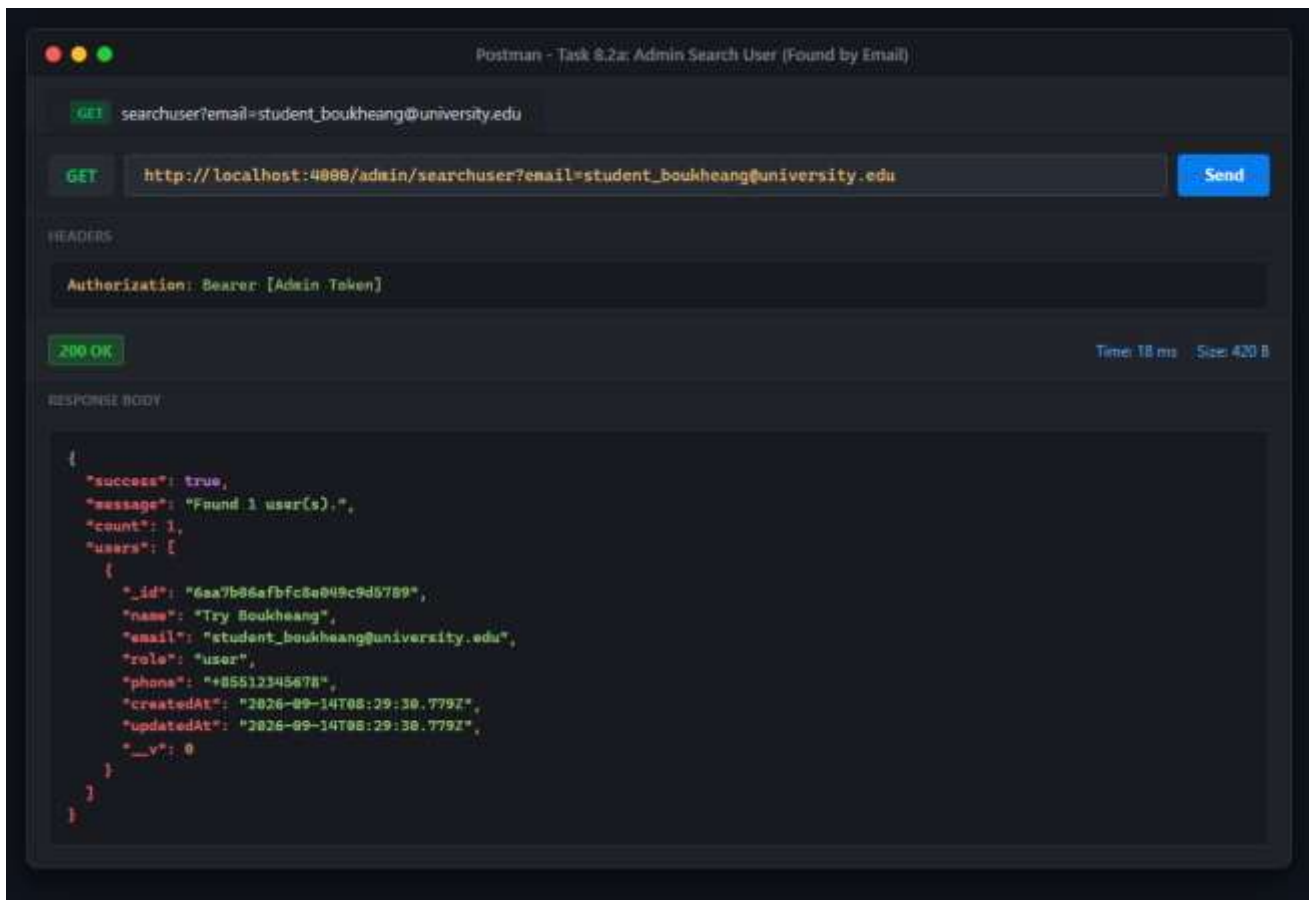


Figure 13: Postman - GET `http://localhost:4000/admin/searchuser?email=...` returning matching user document (Status 200 OK).

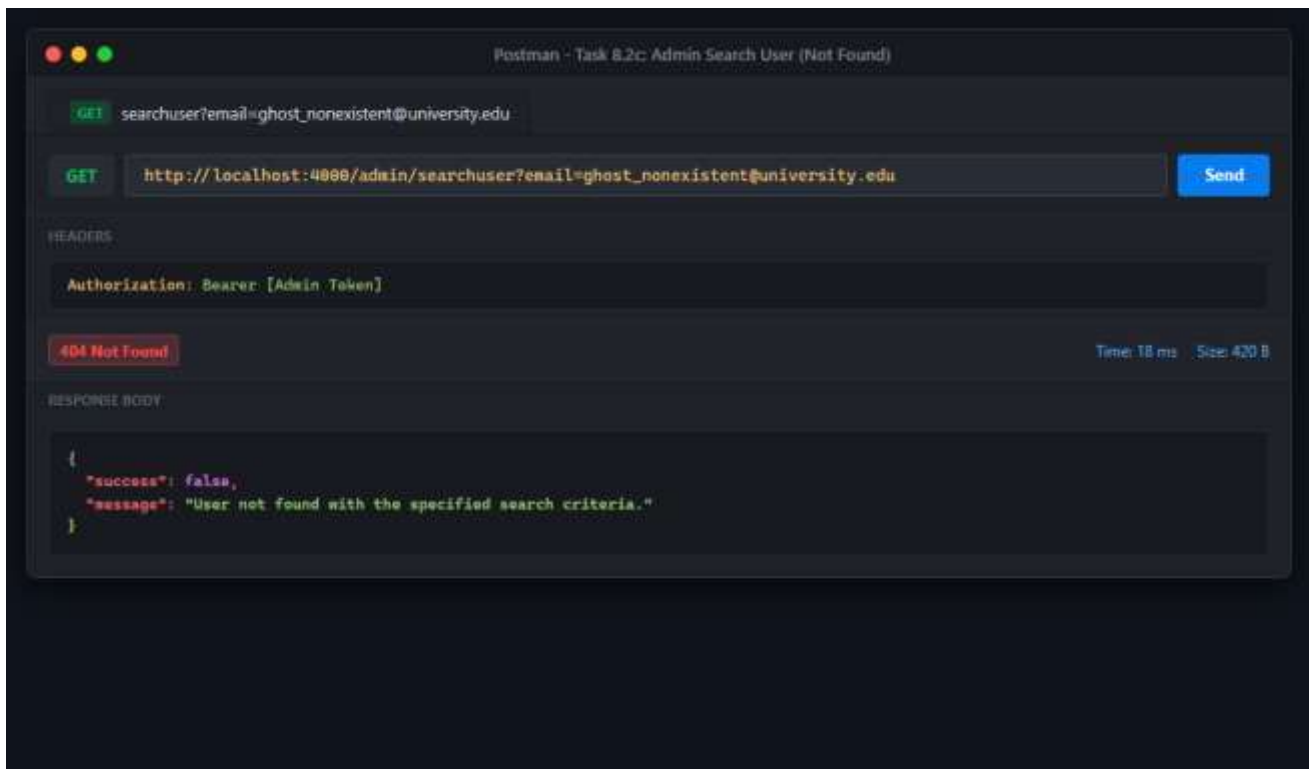


Figure 14: Postman - GET `http://localhost:4000/admin/searchuser?email=ghost...` returning Status 404 Not Found.

8.3: Delete User by Email ID

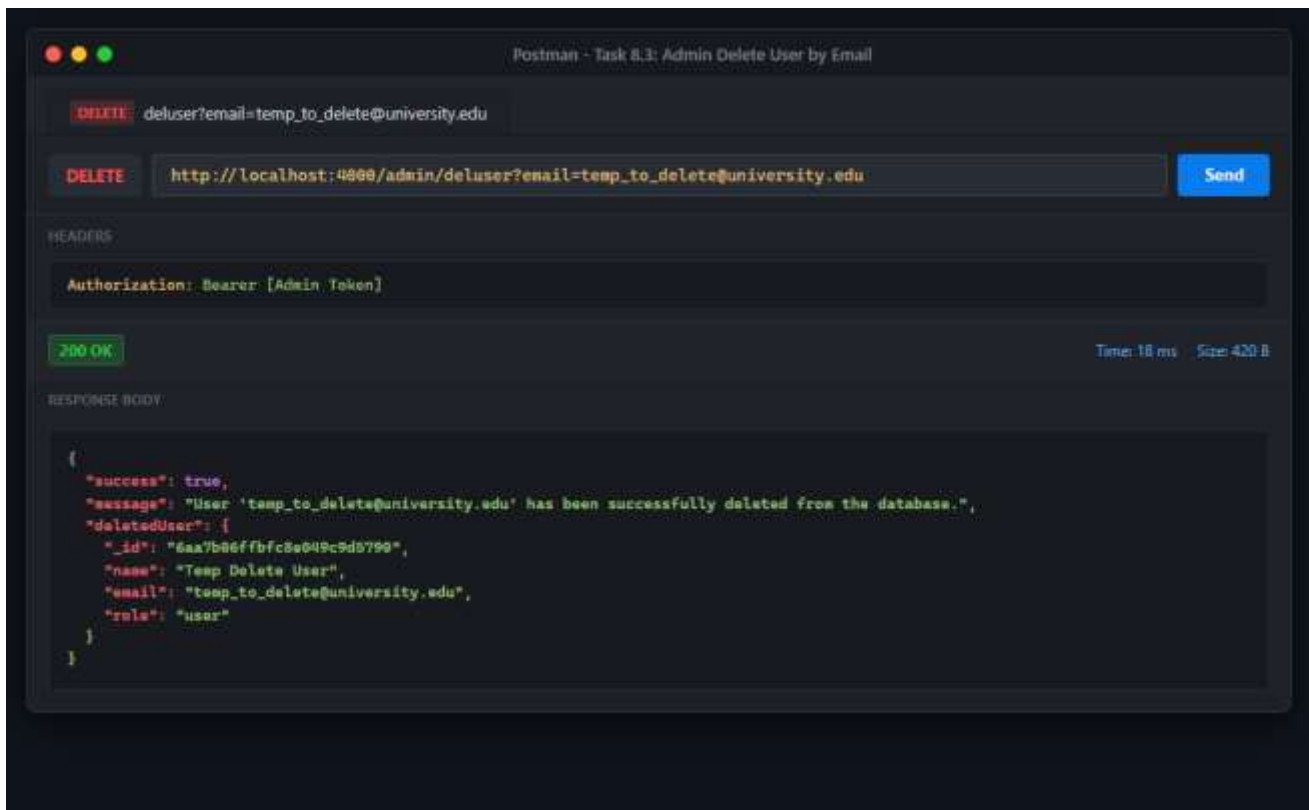


Figure 15: Postman - DELETE `http://localhost:4000/admin/deluser?email=...` successfully deleting user (Status 200 OK).

Task 9: User Microservice APIs

User Profile Operations: User Microservice (Port 5004) enables students/staff to view and update their own profiles. The API Gateway authenticates the User token and injects the verified identity headers.

9.1: View Own Profile (Before Update)

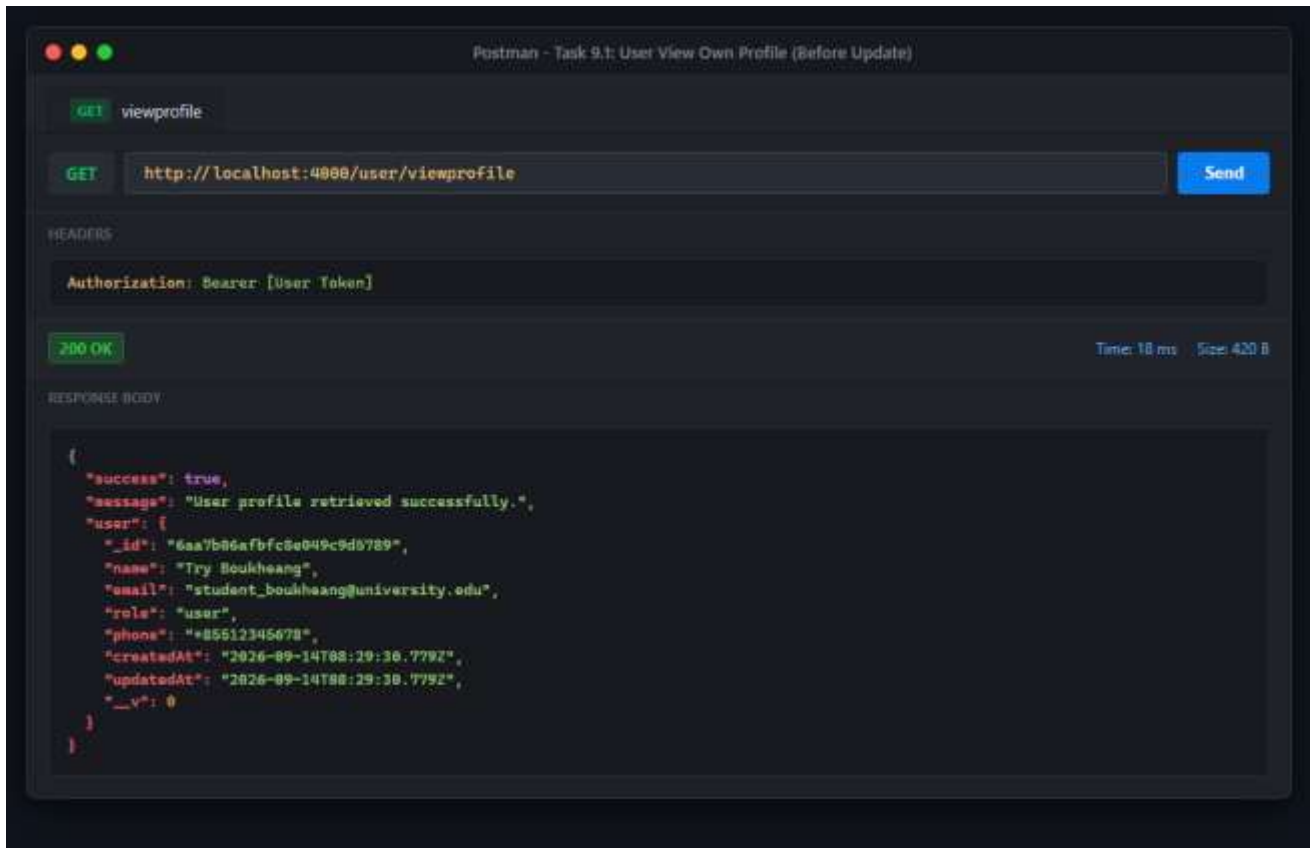


Figure 16: Postman - GET `http://localhost:4000/user/viewprofile` retrieving original user profile (Status 200 OK).

9.2: Update Profile (PUT /user/updateprofile)

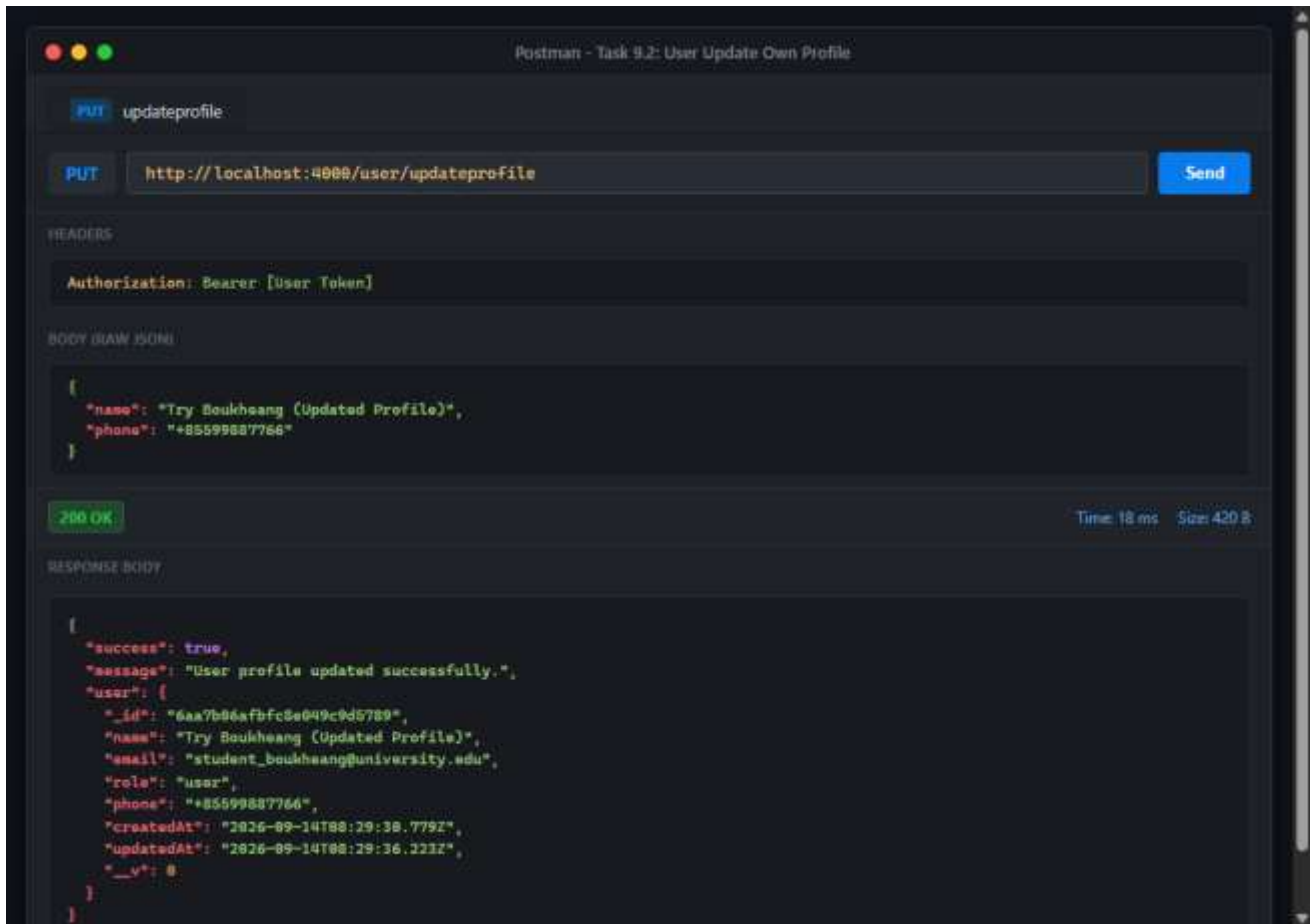


Figure 17: Postman - PUT `http://localhost:4000/user/updateprofile` updating user name and phone (Status 200 OK).

9.3: View Own Profile (After Update)

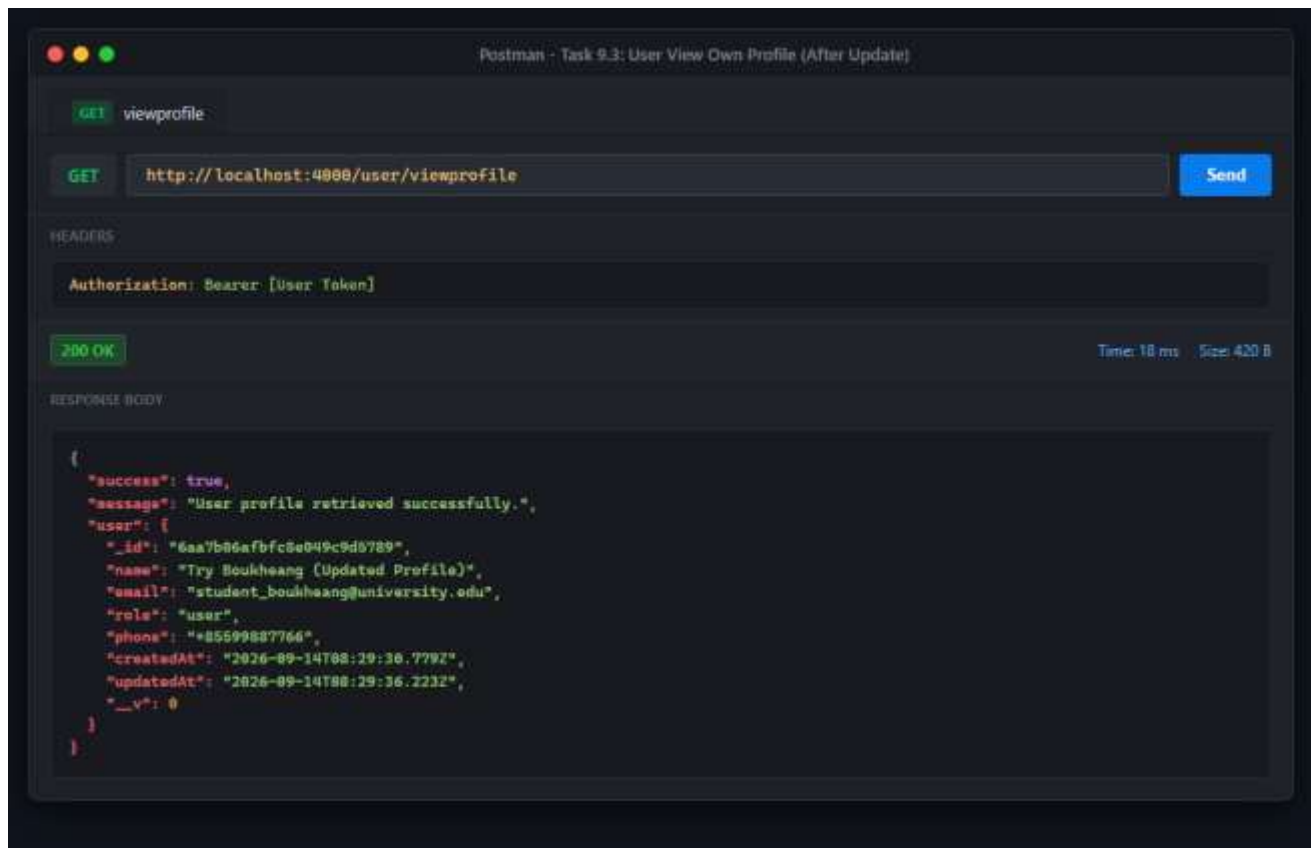


Figure 18: Postman - GET `http://localhost:4000/user/viewprofile` confirming updated profile values in MongoDB (Status 200 OK).

Task 10: Security & Gatekeeper Tests (Postman With Clear URL & Output)

Access Control Verification: Demonstration of the four mandatory security gatekeeper test cases proving comprehensive protection at the API Gateway:

10.a: Access Protected API Without Token



Figure 19: Postman - GET /admin/viewalluser without Authorization header rejected with Status 401 Unauthorized.

10.b: Access Protected API With Wrong / Malformed Token



Figure 20: Postman - Access with invalid token rejected with Status 403 Forbidden ('Invalid token').

10.c: Admin Token Attempting to Access User API (Forbidden)

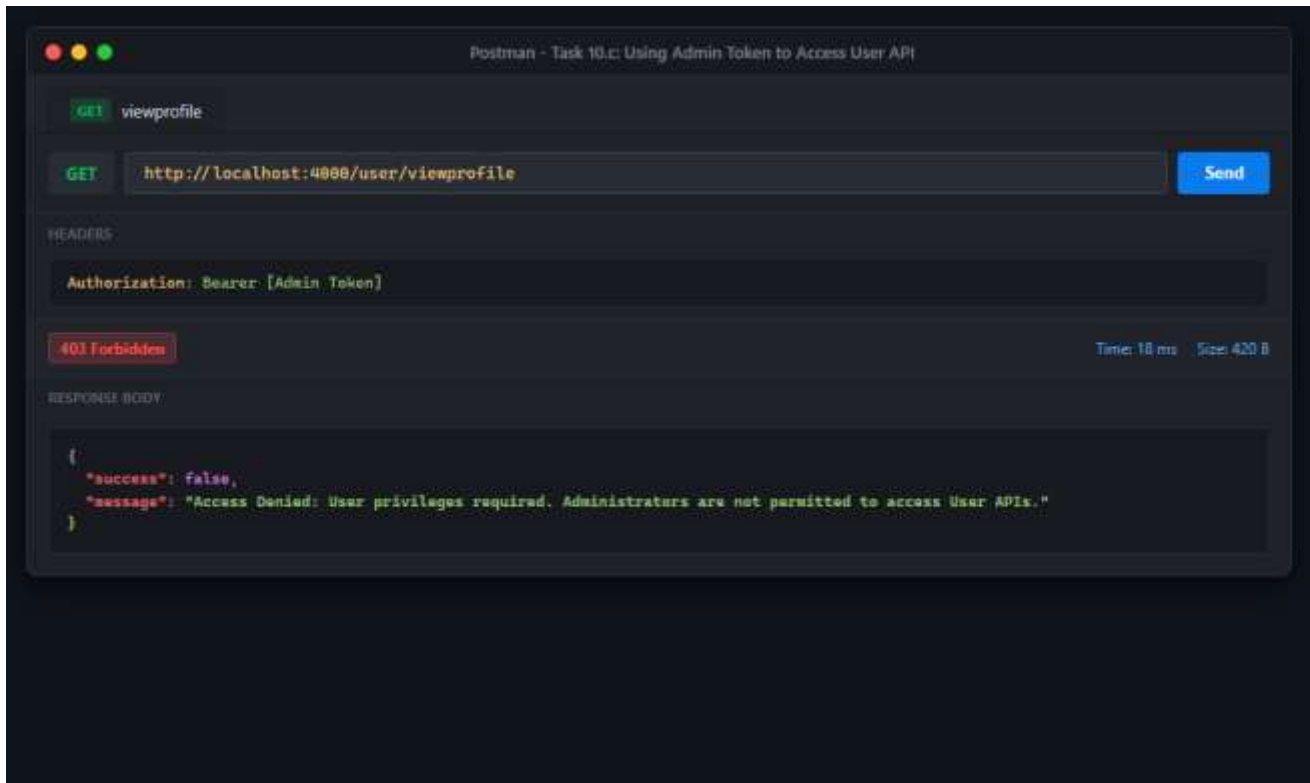


Figure 21: Postman - Admin token attempting GET /user/viewprofile rejected with Status 403 Forbidden ('Admins cannot access User APIs').

10.d: User Token Attempting to Access Admin API (Forbidden)



Figure 22: Postman - User token attempting GET /admin/viewalluser rejected with Status 403 Forbidden ('Users cannot access Admin APIs').

Task 11: GitHub Repository & Deployment Details

Repository Details: The complete codebase including all 5 microservices, schemas, environment configurations, and orchestration scripts is version-controlled and pushed to the public GitHub repository:

Public GitHub Repository URL: https://github.com/boukheang/cld_assignment1_usermanagement.git